

Callback

- A callback is a function passed as an argument to another function.
- A callback function can run after another function has finished.

→ Function Sequence

- Functions are executed in sequence they are called.

```
function myDisplayer(some) {  
    document.getElementById('demo').  
        innerHTML = some;  
}
```

```
function myCalculator(num1, num2,  
    mycallback)
```


{

let sum = num1 + num2;

myCallback(sum);

}

myCalculator(5, 5, myDisplayer);

→ myDisplayer is a callback function.

→ not to use () while passing CB function

→ Call back function are used in async function, where one function has to wait for another function.

→ setTimeout a typical JS async function

→ while using setTimeout(), you can

specify a callback function to be executed on time-out


```
setTimeout(myFunction, 3000);
```

```
function myFunction() {
```

```
document.getElementById('demo').innerHTML  
= 'Hello!';
```

```
}
```

Sync Callbacks

→ Executed immediately
in order

→ Example: Array method
- `map()` or `forEach()`

Async Callbacks

Executed later, after
some operation finish

Example: `setTimeout`,
API calls

Callback Hell

→ Nested callbacks that make code
unreadable

```
setTimeout(Function1)
```



```
setTimeout(Function2) {
```



```
setTimeout (Function3){ }, 1000);  
    }, 1000);  
    }, 1000);
```

```
setInterval ( myFunction , 1000);  
function myFunction() {  
    let d = new Date();  
    document.getElementById('Demo')  
    innerHTML =  
        d.getHours() + ':' +  
        d.getMinutes() + ':' +  
        d.getSeconds();  
}
```


Promises

→ Producing Code is code that can take some time

→ Consuming Code is code that must wait for the result.

→ A promise is an Object that links Producing code and Consuming Code.

```
let myPromise = new Promise
```

```
(function (myResolve, myReject)
```

```
{
```

```
  myResolve(); // success
```

```
  myReject(); // error
```

```
});
```

```
myPromise.then(
```

```
  function (value) { // if success },
```


function (error)

```
{ // if error },  
};
```

Promise Object Properties

ks A JS promise object can be:

- le. → Pending
- Fulfilled
- Rejected

ed) The promise Object supports two properties:

- State
- Result

→ While Promise Object is 'pending', the result is undefined.

ss },

When a Promise Object is "fulfilled", the result is a value.

When a Promise object is "rejected" the result is an error object.

myPromise.state

"pending"

"fulfilled"

"rejected"

myPromise.result

undefined

a result value

an error object

How to use Promise :

→ `Promise.then()` takes two arguments a callback for success and another for failure.

→ Both are optional, you can add a callback for success or failure only

- `then()` → runs on success
- `catch()` → runs on failure
- `finally()` → always runs (cleanup)

Chaining Promises

`fetch("https://jsonplaceholder")`

- `then(res ⇒ res.json())`
- `then(data ⇒ console.log(data))`
- `catch(err ⇒ console.errorlog(err))`
- `finally(() ⇒ console.log('Done'));`